Rabbit Reorder and Potential Enhancements

Russell C

Problem Space

Memory Access Pattern of Graph Traversals and SpMV

CSR SpMV sequentially accesses arrays *except* **x**!

There are irregular indirect accesses to x through A_C

- Consecutive accesses may be to x[2], x[4], x[7], etc.
- This exhibits poor locality
- This is why SpMV suffers from cache misses; no notion of predictive caching

Algorithm 1 Sparse matrix-vector multiplication

Input: $n \times n$ CSR matrix $A = (A_V, A_C, A_I)$ and vector $\boldsymbol{x} \in \mathbb{R}^n$
Output: $\boldsymbol{y} = A\boldsymbol{x}$

```
1  for v = 0 to n − 1 do
2      y[v] ← 0
3      for k = A_I[v] to A_I[v + 1] − 1 do
4          y[v] ← y[v] + A_V[k] x[A_C[k]]
```

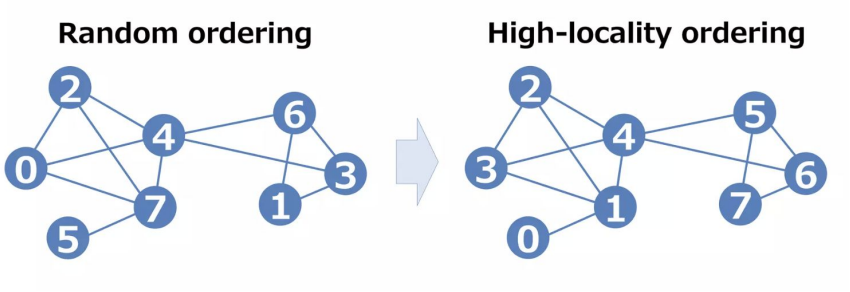
Line 4: irregular accesses to elements of x in the summing step

Poor Locality, Spatial and Temporal

Randomly ordered graphs are limited in terms of “local vertex reach.”

Low locality implies:

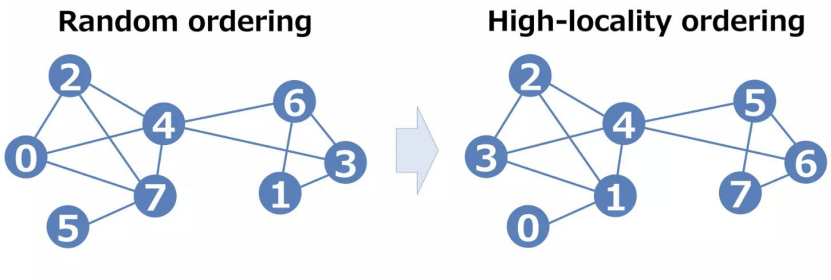
- Spatial: vertices which could be clustered and loaded into a appropriate cache level are far apart in memory → *cache misses*
- Temporal: unstructured graphs need more time scanning a graph, due to lack of presence in cache



Graph Reordering

In the field of Graph Analysis:

Graph reordering is used to improving access patterns of vertices in traversals



$$\underset{B}{\begin{bmatrix} 1 & 0 & 3 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 2 & 4 & 0 \\ 7 & 8 & 0 & 0 \end{bmatrix}} \times \underset{X}{\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}} = \underset{A}{\begin{bmatrix} 10 \\ 0 \\ 16 \\ 23 \end{bmatrix}}$$

Graph and its Induced Adjacency Matrix

We can form an adjacency matrix from a graph, and vice versa, such that:

- Nodes in the graph correspond to the rows/columns of the matrix
- Edges in the graph corresponds to the non-zero entries in the matrix

What can we do interpreting a (square) matrix as the adjacency matrix for a corresponding induced graph?

Why not Reorder?

- Effective reordering calls for high computational cost to generate such an ordering
- Reordering is typically orders of magnitude slower than the operation at hand
  - i.e. reordering A runtime >> SpMV
- Increases end-to-end operation time, need high usage to amortize cost
- Higher complexity reordering schemes typically grow faster than n^2

Philosophy

If a given problem interacts with a large sparse matrix enough, either in SpMV or SpMM, it is worth to tank reordering time for much better long term performance in said operations.

Problems like this:

- Graph algorithms: PageRank score computation, social network analysis
- Iterative solvers: finite element analysis, computational fluid dynamics

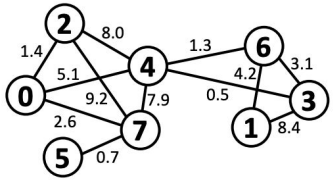
Community-Based Reordering

What are Communities?

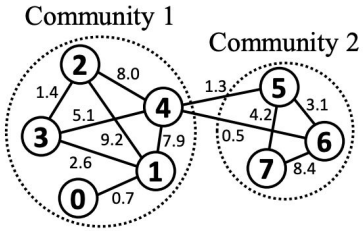
Communities of vertices have dense count of intra-community edges

<u>Key fact:</u> *densely grouped* communities imply close row and column spatial proximity between community vertices.

These edges map to dense, per-community “region” structure.



(a) Randomly ordered graph



(b) Reordered graph

	0	1	2	3	4	5	6	7
0			1.4		5.1			2.6
1				8.4			4.2	
2	1.4				8.0			9.2
3		8.4			0.5		3.1	
4	5.1		8.0	0.5			1.3	7.9
5								0.7
6		4.2		3.1	1.3			
7	2.6		9.2		7.9	0.7		

(c) Adjacency matrix of graph (a)

		0	1	2	3	4	5	6	7
Community 1	0		0.7						
	1	0.7		9.2	2.6	7.9			
	2		9.2		1.4	8.0			
	3		2.6	1.4		5.1			
	4		7.9	8.0	5.1		1.3	0.5	
Community 2	5					1.3		3.1	4.2
	6					0.5	3.1		8.4
	7						4.2	8.4	

(d) Adjacency matrix of graph (b)

General Method Overview

- Graphs contain n nodes/vertices which have pairwise edges with an associated edge

We want to be able to give an algorithm a sparse matrix A to reorder:

- The algorithm will view A as an adjacency matrix on n vertices
- Perform clustering/grouping of vertices into **communities** which manifest as blocks in the outputted permuted P(A) adjacency matrix

$$P(A) := PAP^T$$

Implications of Reordering

- Reordered graphs allow neighboring vertices to be grouped together
  - Grouped communities of vertices in the permutation imply grouped rows and columns!
- Reordered graphs are associated with dense region-containing adjacency matrix:
  - a more persistent caching of relevant data (high temporal locality) and
  - by the density of said data in memory (high spatial locality)

$$(\text{ID vector})\ [2,0,3,1] \rightarrow \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} =: P$$

Rabbit Order (2016)

Community Modularity and How to Choose a Merge

Modularity Q of a community is used as a metric of “partitionedness” of a graph:

$$Q_c = \frac{1}{2m} \sum_i \sum_j A_{ij} \mathbf{1}\{c_i = c_j = c\} - \left(\sum_i \frac{k_i}{2m} \mathbf{1}\{c_i = c\} \right)^2$$
$$= \frac{\Sigma_{in}}{2m} - \left(\frac{\Sigma_{tot}}{2m} \right)^2$$

We query a vertex’s set of edge-neighbors and check which merge increases modularity the most:

$$\Delta Q(u, v) = 2 \left\{ \frac{w_{uv}}{2m} - \frac{d(u)d(v)}{(2m)^2} \right\}$$

Core Approaches

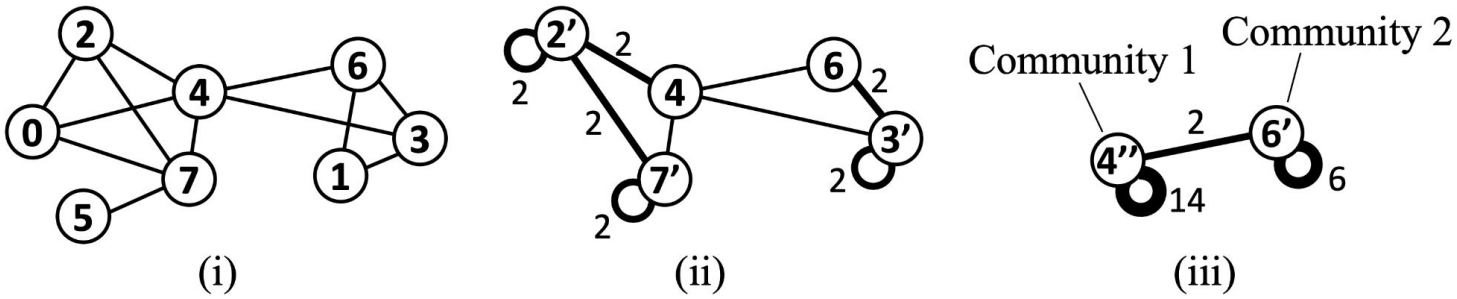
Incremental Community Aggregation

- Shortens reordering time by merging vertices into a common community in parallel
  - Use dendrogram to track path to route and new sub-merges
- This coarsens the graph
- Decreases the number of vertices to processes

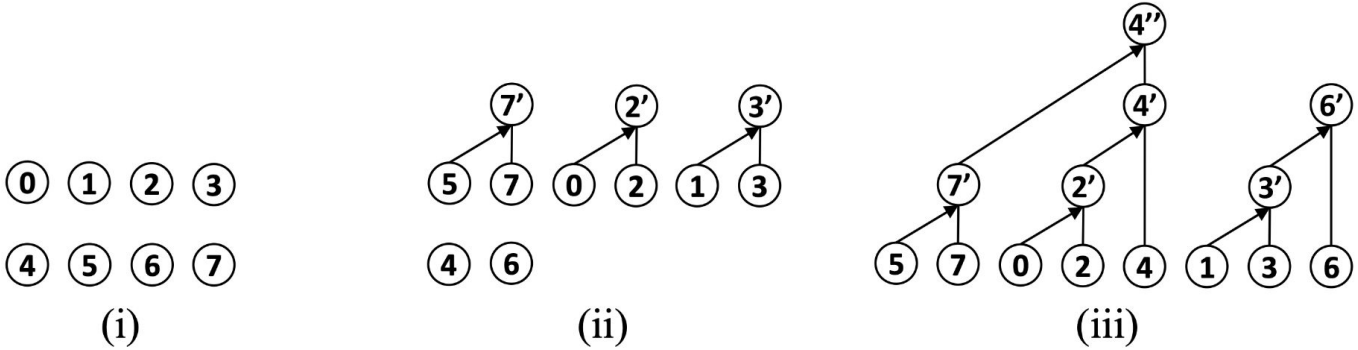
Hierarchical Community-Based Ordering

- Starting at top level and recurring over extracted communities, detect communities by doing ICA
- Communities at varying levels may hope to be accommodated in a suitable cache level
- Yields a new ordering optimal up to the modularity heuristic Q

Community Detection



(a) Incremental aggregation on the graph



(b) Dendrogram construction

Hierarchical Community-Based Ordering Algorithm Steps

Perform Community Detection

Extract hierarchical communities in an agglomerative manner while updating an implicit dendrogram at each merge

Perform Ordering Generation

Extract, from the dendrogram, a new ordering such that:

> **for every community hierarchy, vertices in the same community are co-located**

Repeat until a maximum community level (hyperparameter) is achieved

Incremental Community Aggregation

- Incrementally aggregate suitable vertices into the same community to coarsen graph
- Each merge of a neighbor *source* vertex into a new *destination* vertex (representing the community) forms a cumulative weighted edge between the two
- Repeated merging extracts hierarchical communities while maintaining our corresponding dendrogram for path compression

Key Contributions of Rabbit (2016)

1. High locality: exhibited locality beats or matches state-of-the-art methods
2. Fast reordering: short runtime; outperforms said methods
3. Extensive evaluation: compared with seven other reordering methods
    a. Slashburn (2014)
    b. Reverse Cuthill-Mckee (unordered parallel RCM, 2014)
    c. Shingle ordering (2009)
    d. BFS (unordered parallel BFS, 2014)
    e. Layered Label Propagation (2011)
    f. Multithreaded Nested Dissection (2013)
    g. Degree: increasing order of vertex degree

Serial Algorithm

Vertex Constructor

Initialize self attributes:

1. strength = strength
2. child = None
3. sibling = None
4. united_child = None

Edge Constructor

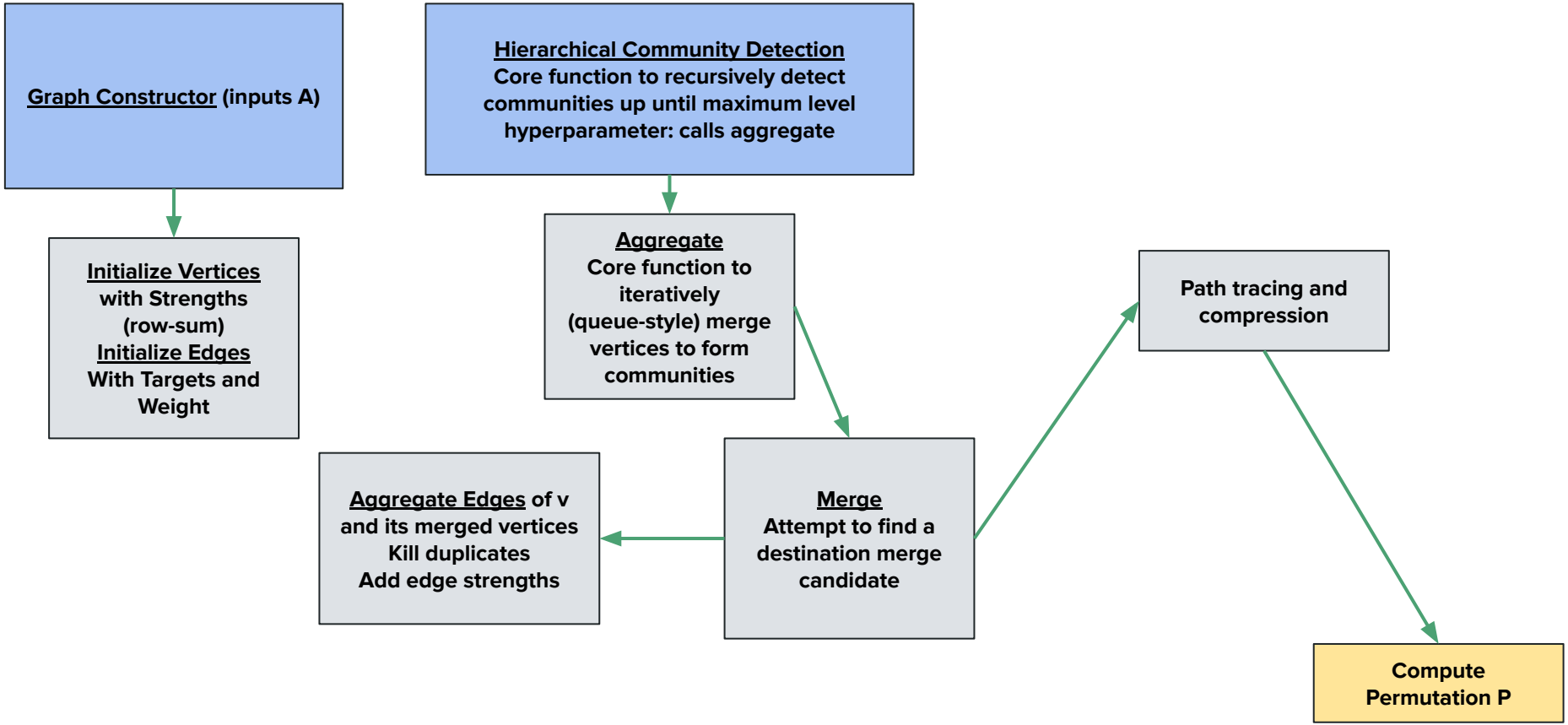
Initialize self attributes:

1. target = target
2. weight = weight

1. **Vertex strength**: aggregate weight of its edges, the row sum of the associated vertex
2. **Child**: ID pointing to the last vertex merged into this vector
   - Induces representative-representee relation
3. Sibling: next vector in list of merged vertices
   - Used to induce a linked list of merges
4. United_child: last united child
   - Used for tracking unites to avoid redundant operations

Unilateral Edge Structure

1. Target: pointer to target vertex this edge belongs to
2. Weight: associated edge weight, as in adjacency matrix


Graph Constructor (inputs A)
Hierarchical Community Detection
Core function to recursively detect communities up until maximum level hyperparameter: calls aggregate
Initialize Vertices with Strengths (row-sum)
Initialize Edges With Targets and Weight
Aggregate
Core function to iteratively (queue-style) merge vertices to form communities
Path tracing and compression
Aggregate Edges of v and its merged vertices
Kill duplicates
Add edge strengths
Merge
Attempt to find a destination merge candidate
Compute Permutation P

Graph Constructor (inputs `adj`)

Initialize self attributes:

- `edges` = adj
- `communities` = empty arrays
- `total_weight` = 0.0

Call `initialize_vertices`

Initialize `num_vertices` = # vertices

initialize_vertices

num_vertices = # of edges in adj
communities = sequentially ID each vertex with its own community

For each v_id:

- strength = total edge weight [row_sum]
- initialize Vertex(strength) into Graph.vertices; accumulate strength into total_weight

trace_com

Search `Graph.communities`

- If own community, return
- Else, keep going deeper in community level

If v's community ID, `v_id` in `Graph's communities` array not updated, set to found community
return `com_id`

compact

Aggregate duplicate edges in `Graph.edges` by summing weights, return new edge list

unite

Aggregate edges of `v` and its merged vertices

build_compressed_graph

Builds compressed graph where each node represents a community

merge

Attempts to merge Vertex v into neighbor
Call unite, call find_best
If found best, update destination u's strength, sibling as old child, child as v
Update community of v_id as u_id

find_best

Find neighbor with greatest (+) modularity gain, return input v_id if none

aggregate

Core function to iteratively merge vertices to form communities
In order of ID, init `pending` as empty array

- for `w` in pending, attempt `merge`, if success append `w` to `pending`

Attempt `merge` on `v`
Attent `merge` on all `w` in remaining `pending`

get_communities

Extract communities from final community assignments
For each `v`:

- `c` = `trace_com` of `v`
- if c not in community_map, set value to empty array in community_map for key c
- Append value v to key c

return community_map

compute_perm

Compute permutation of the vertices based on community assignments
For each v in community_map, Perm[v] = new_id, accumulate new_id
reset accumulator new_id

hierarchical_community_detection

Core function to recursively detect communities

Start at level 0
current_graph = self
While level < max_level:

- call aggregate current_graph
- compressed_graph = `current_graph.get_communities`
- if num_vertices is stable, set current_graph to the compressed_graph increment

adjacency_list := adj

Rabbit++ (2023; V. Balaji, N. C. Crago, A. Jaleel and S. W. Keckler)

Insularity and Hub Group & Sort: Post-Community Detection

+**Insularity** is defined as #intra-community edges / #outgoing edges

- Higher, the more “connection-lodged” or embedded a vertex is in its community
- Can group non-insular nodes last (anywhere OTHER than where we group “good” nodes)

+**Hubs** are vertices that surpass the threshold: out-degree > param * average degree

- Extension of community representative definition within community
- Higher, more heavy in relative connectivity; Degree
- We may sort them to increase effectiveness of this prioritization

Ongoing Work

Idea: Stacking

Rabbit ordering improves clumpiness of the matrix, and thus improves memory access pattern of SpMV by keeping highly accessed, neighboring entries of x for longer times in cache.

However, community-based methods are expensive to run, high locality -> slow reordering. BFS-based methods are cheap and heuristic.

What if we stack a BFS method in pre processing or post processing?

Idea: Blocked Reordering

Rabbit is expensive in terms of dimension-scaling.

Can we block a matrix and still reap speedup?

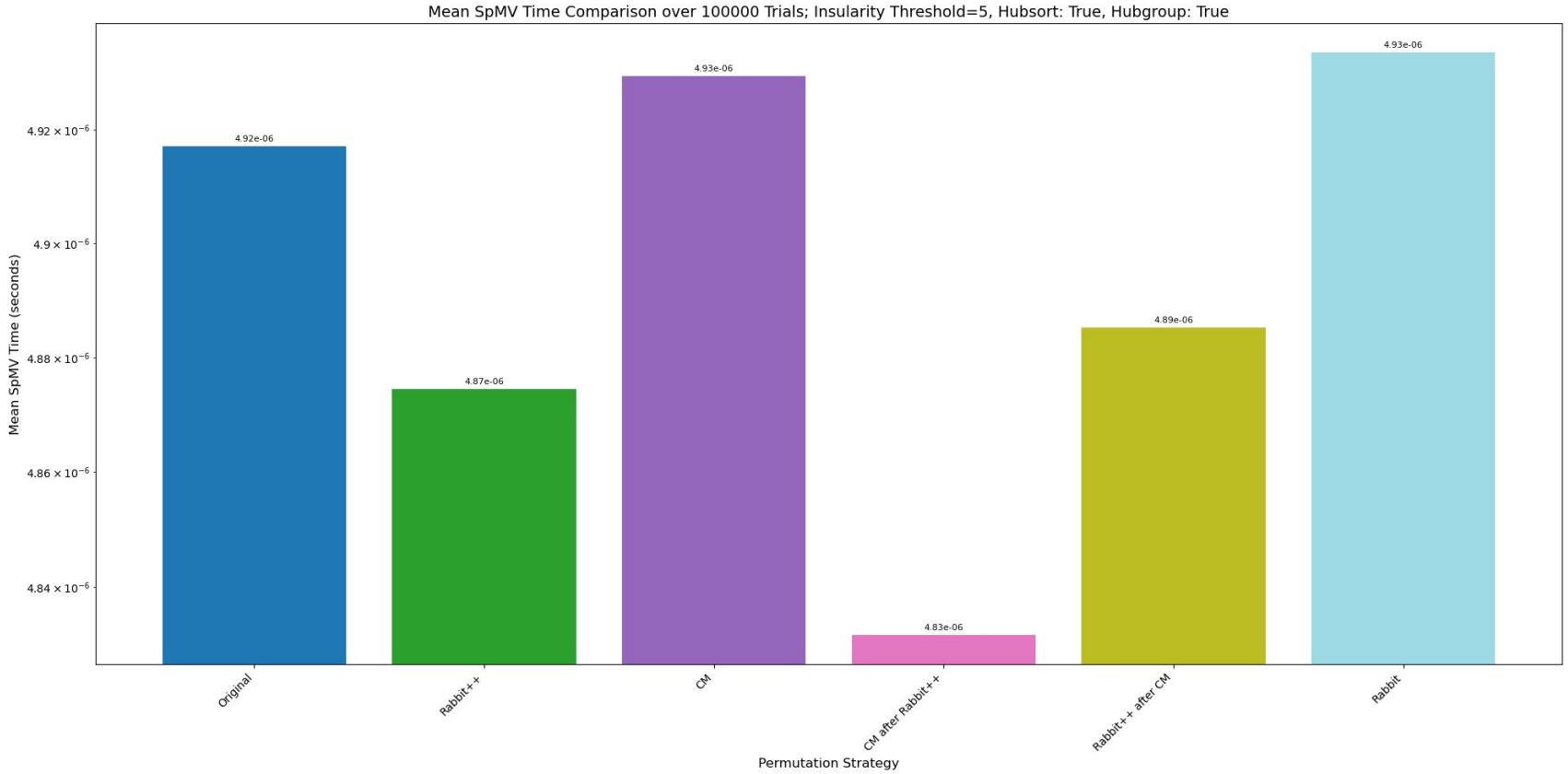
Idea: Can we Optimize SpMV for Community-based Blocking?

Since an induced hierarchy is returned by the process, can we explicitly assign communities to be blocked into caches, given cache sizes, for a speedup?

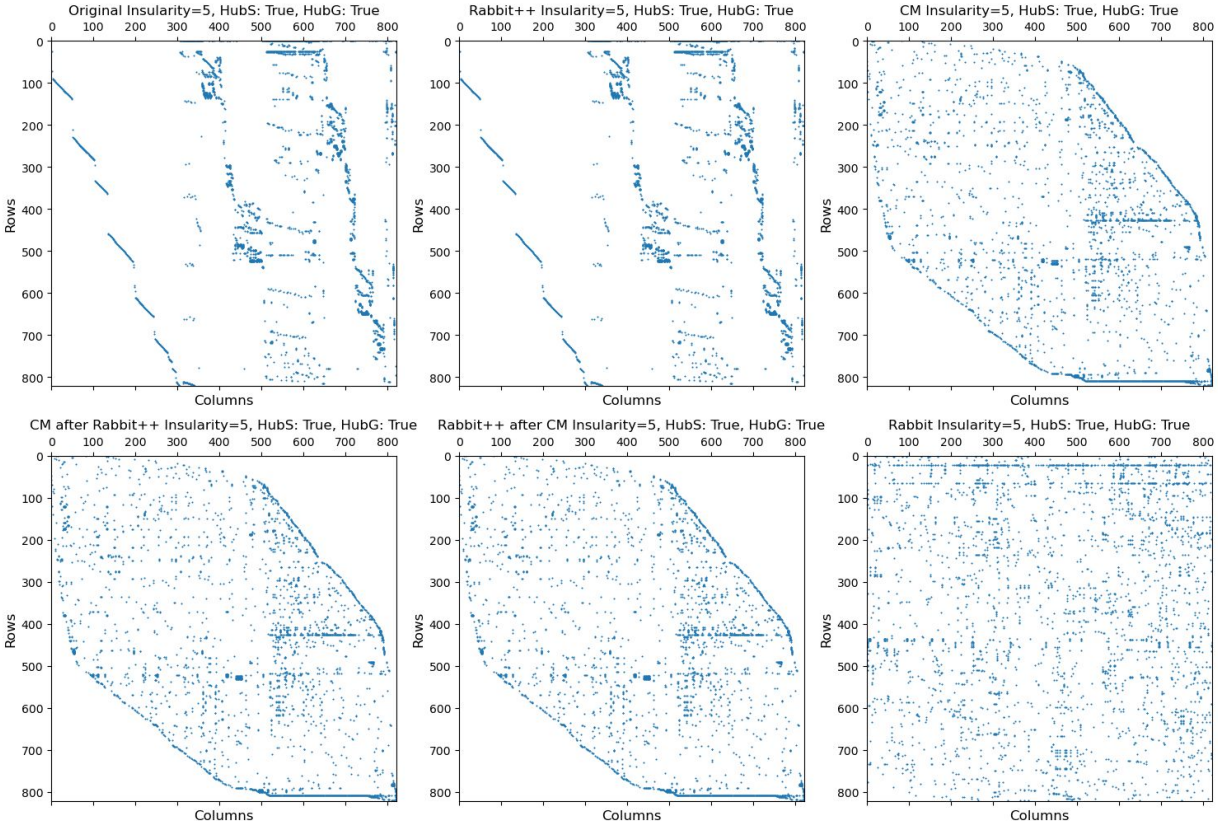
Is there further work to be done in per-block analysis of reordering regimes?

SpMV Runtimes, Favorable (clustered, non-structured)

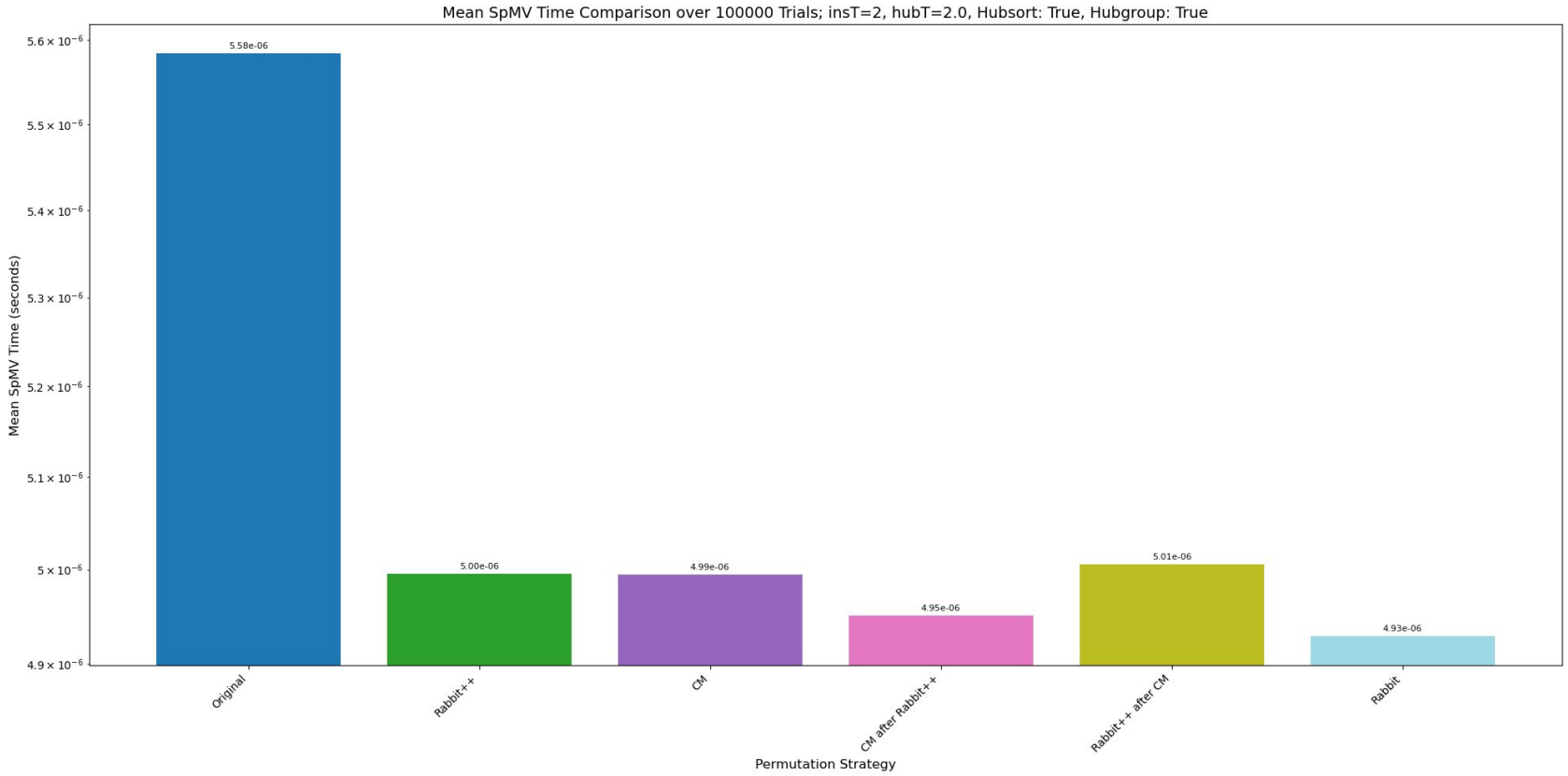
bp_0: 822-dimensional Optimization Problem Sequence



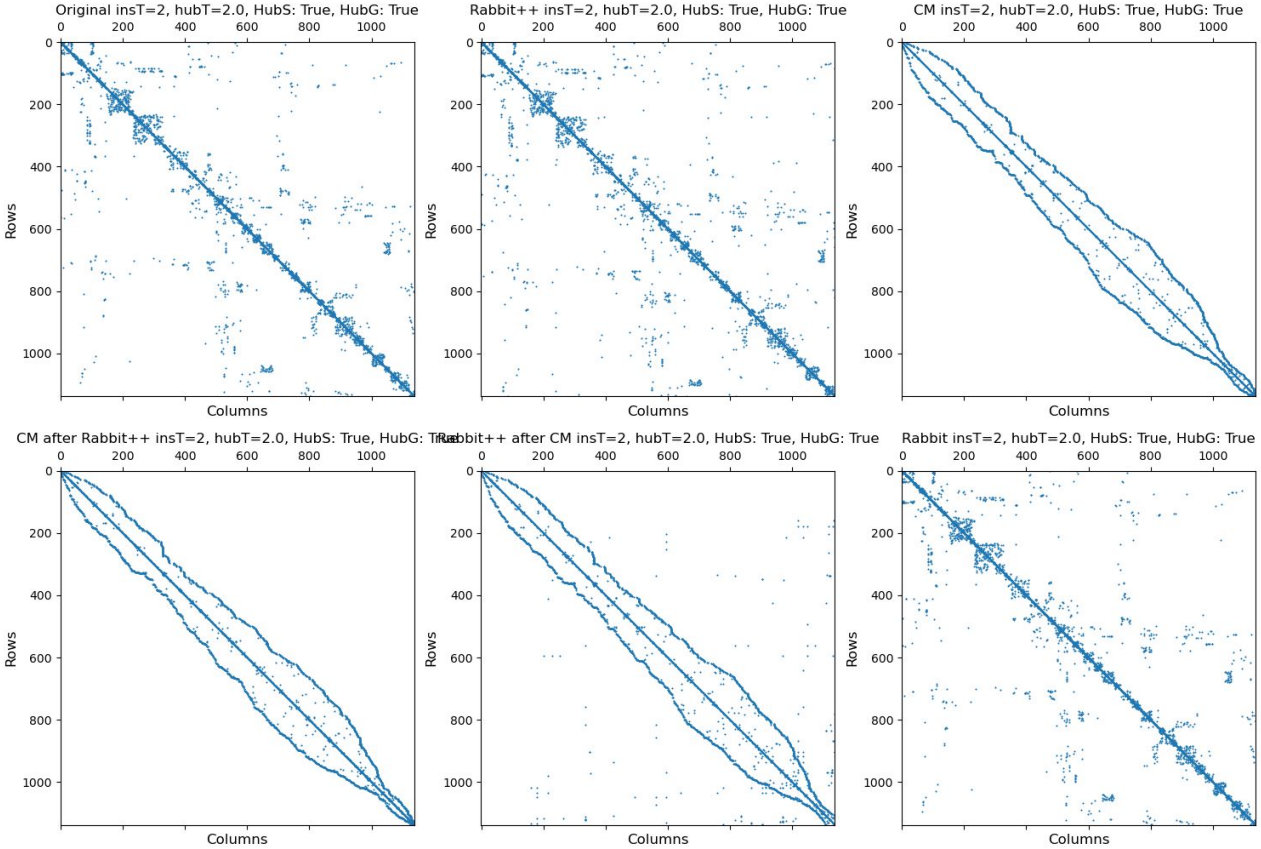
Associated Spy Plots



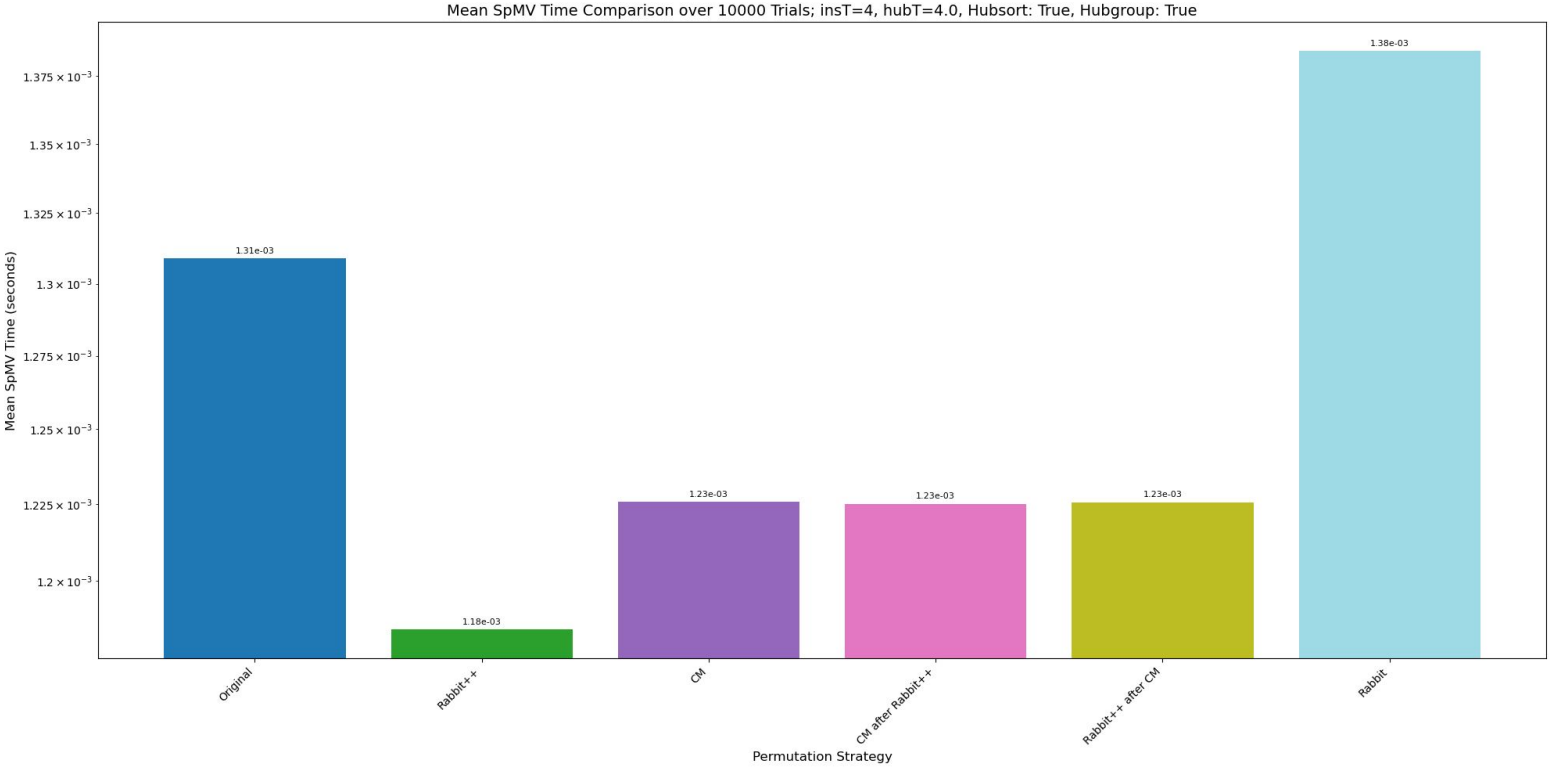
1138_bus: 1138-dimensional Power Network Problem



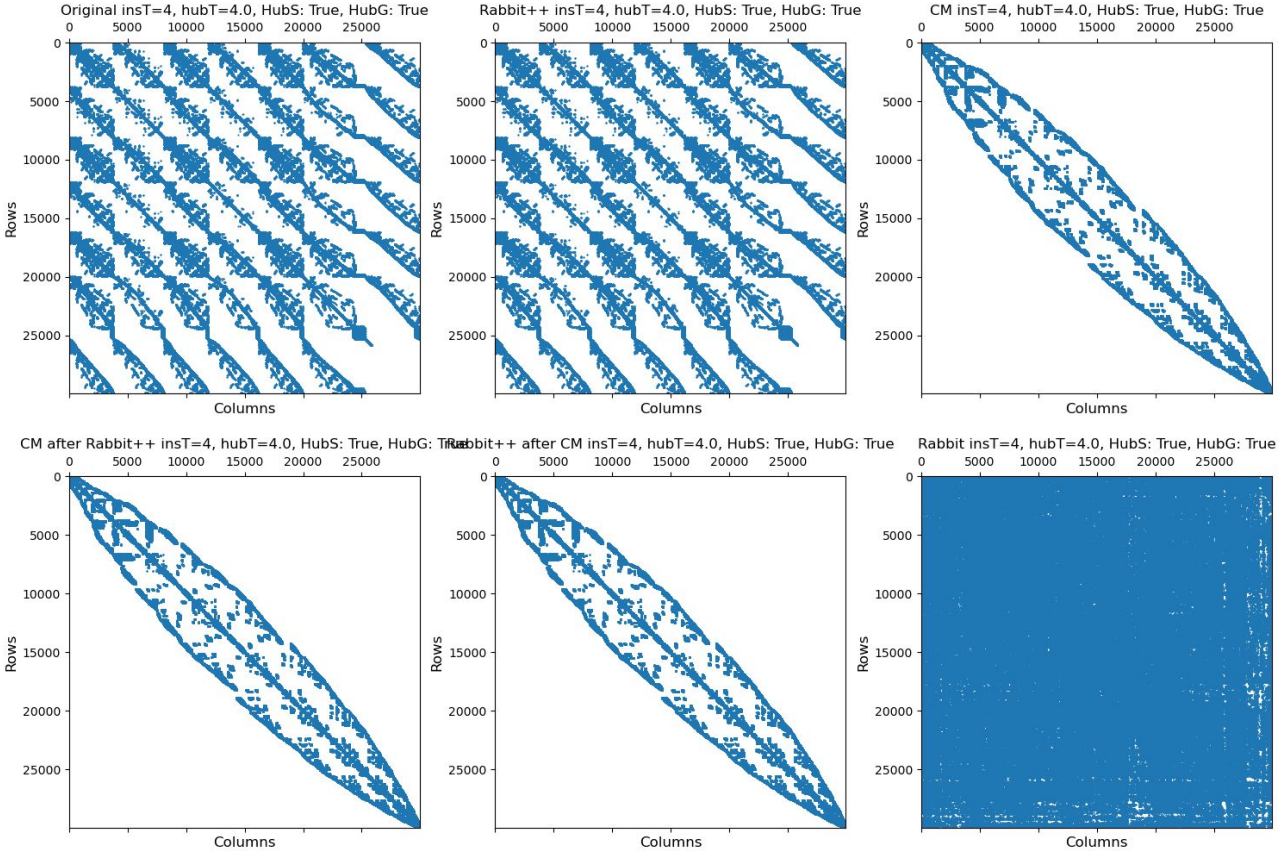
Associated Spy Plots



mixtank_new: 29957-dimensional Computational Fluid Dynamics Problem (ins=4.0, hub=4.0)



Associated Spy Plots



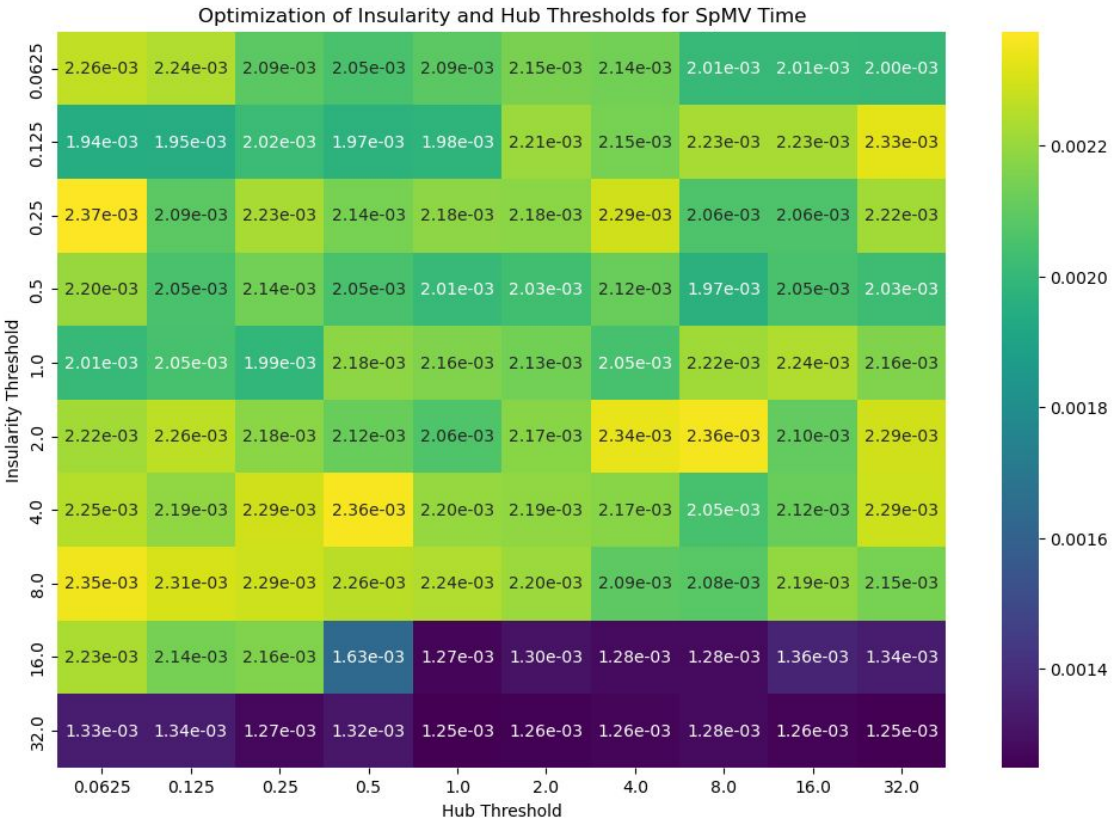
Stacking does <u>not</u> help here!

The original matrix already had blocky structure! Can we optimize over Rabbit++'s parameters, insularity and hub threshold, to beat Rabbit++?

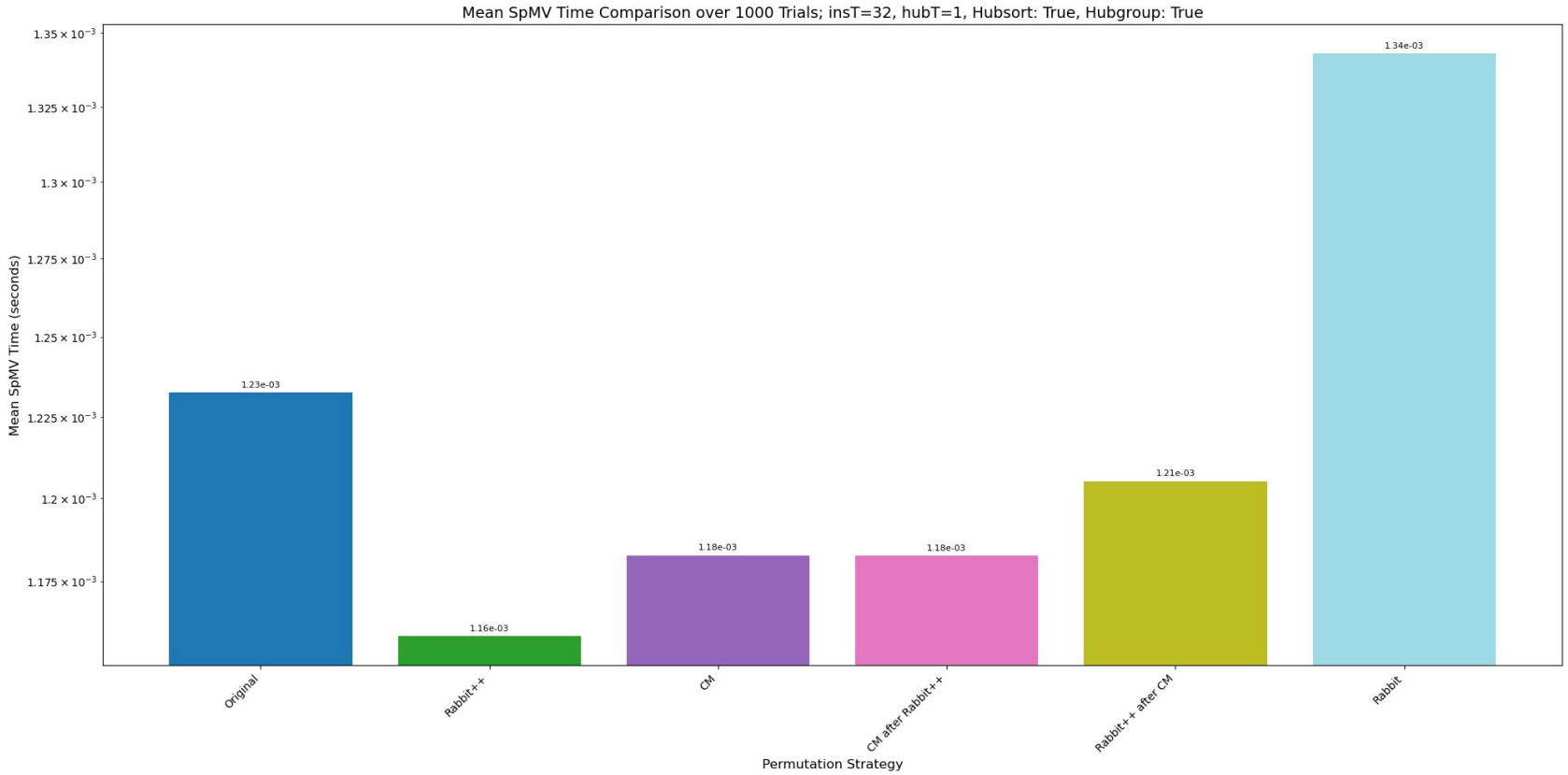
Let us try to optimize over this 2-d space for Cuthill-Mckee after Rabbit++:

1. Insularity Threshold: from 2^(-4) to 2^5; 0.0625 to 32
2. Hub Threshold: from 2^(-4) to 2^5; 0.0625 to 32

Searching over 2-d space…

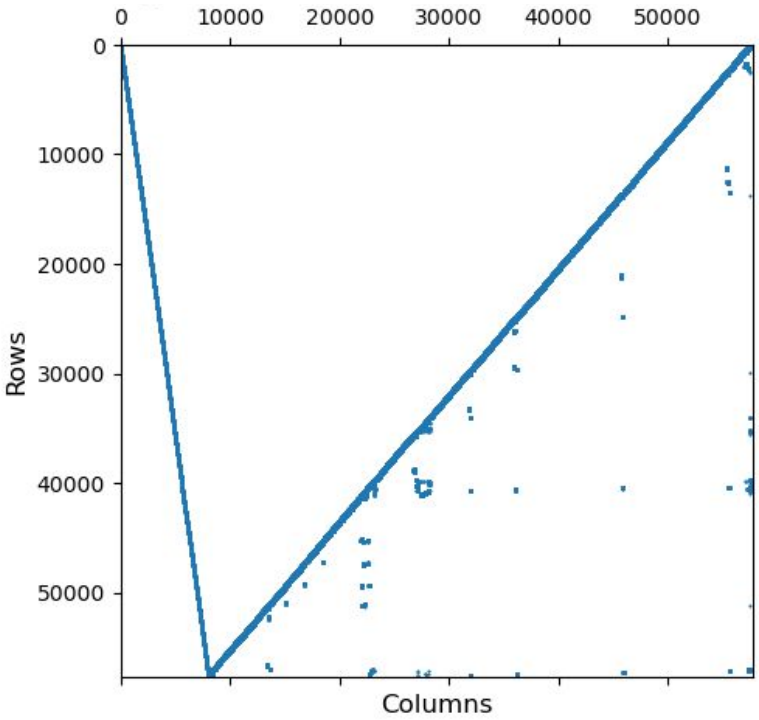


After optimizing parameters for Rabbit++ after CM… (all rabbit++ uses same params): **2% Rabbit++, 4% CM after Rabbit++ improvement**

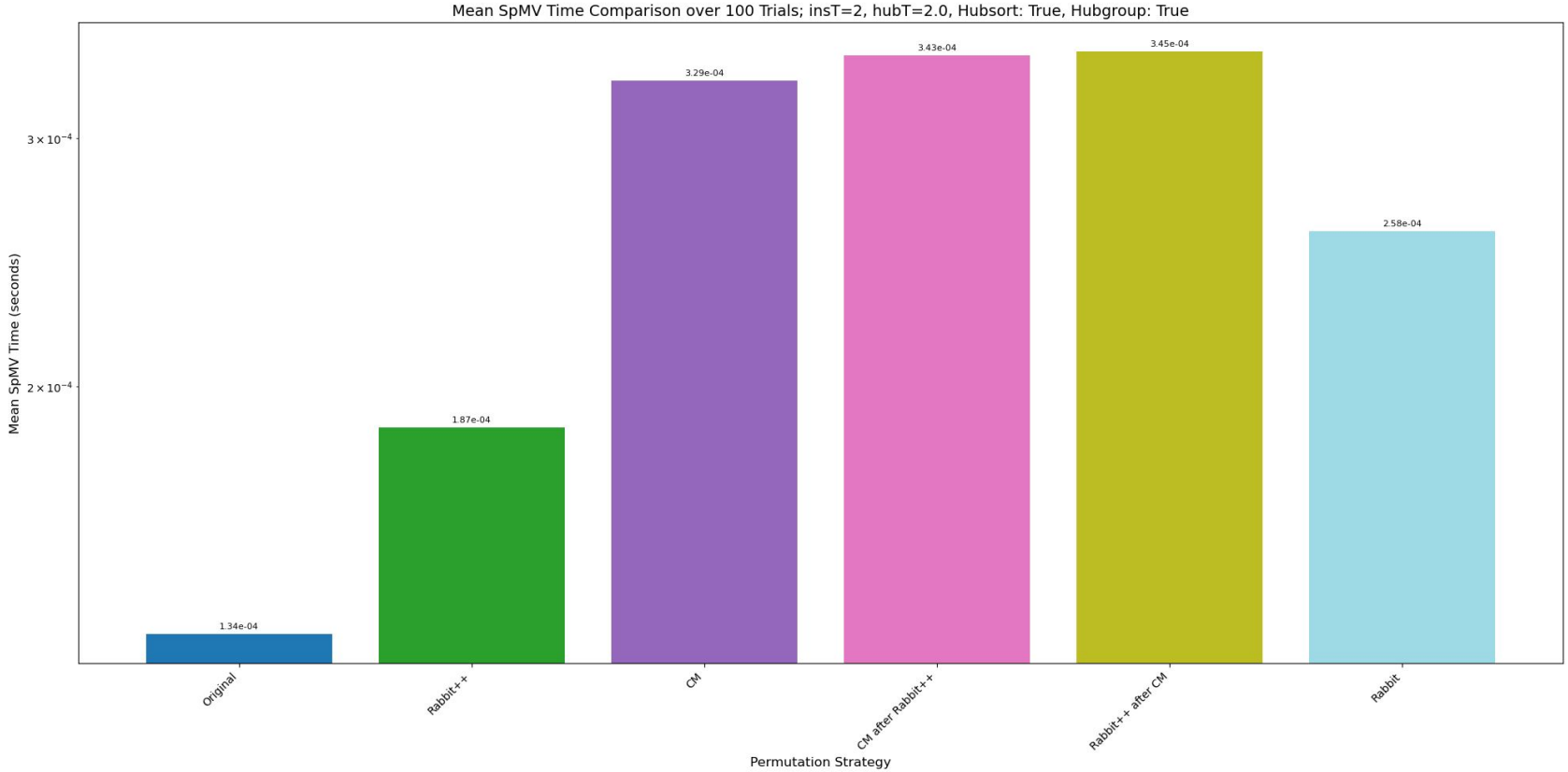


SpMV Runtimes, Adversarial (strongly [quasi-] diagonal structure)

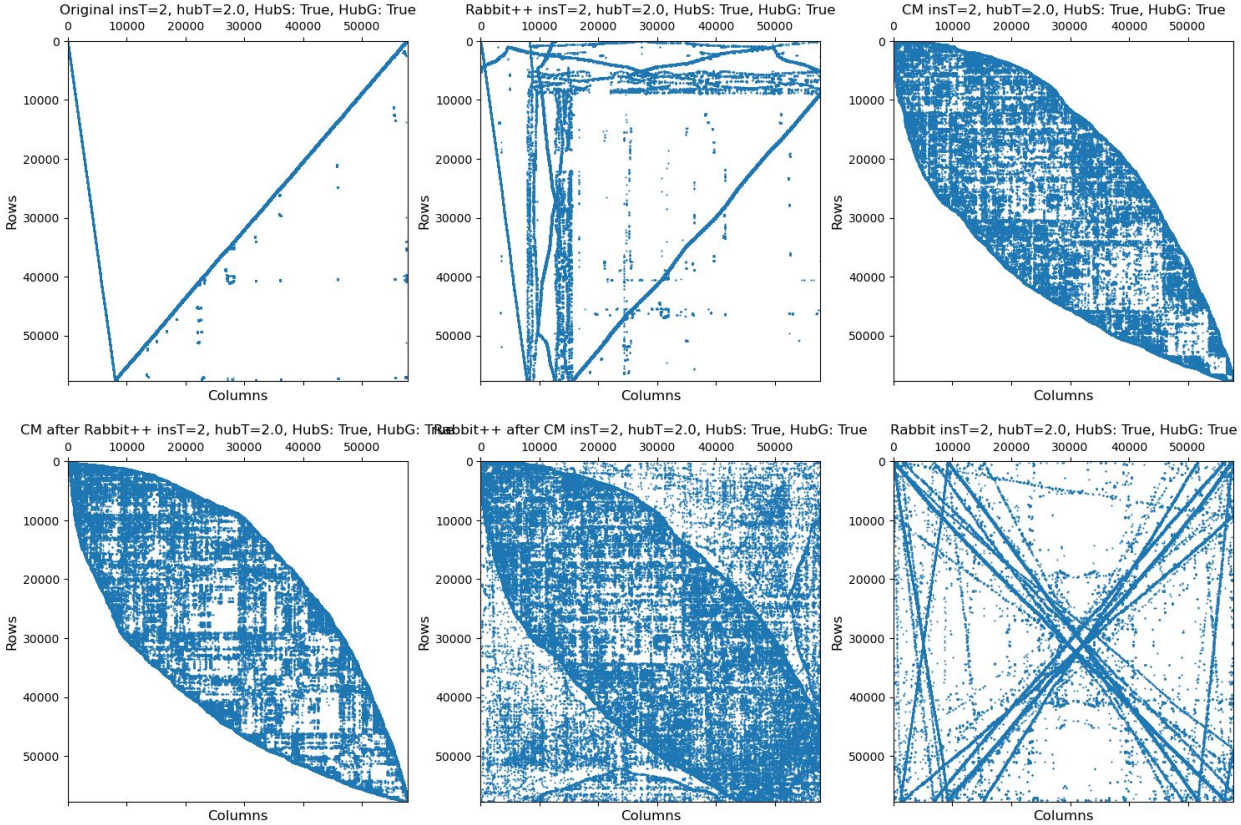
bayer01: 57735-dimensional Chemical Process SimProblem



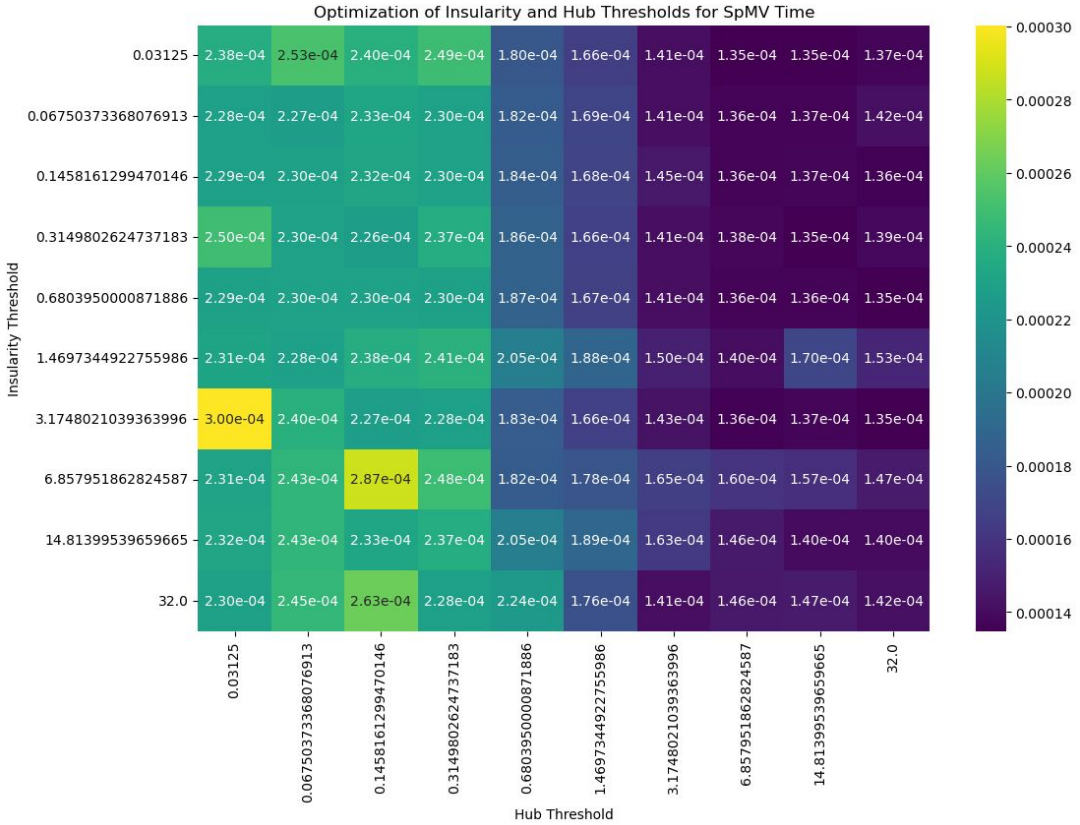
bayer01: Stacking SpMVs



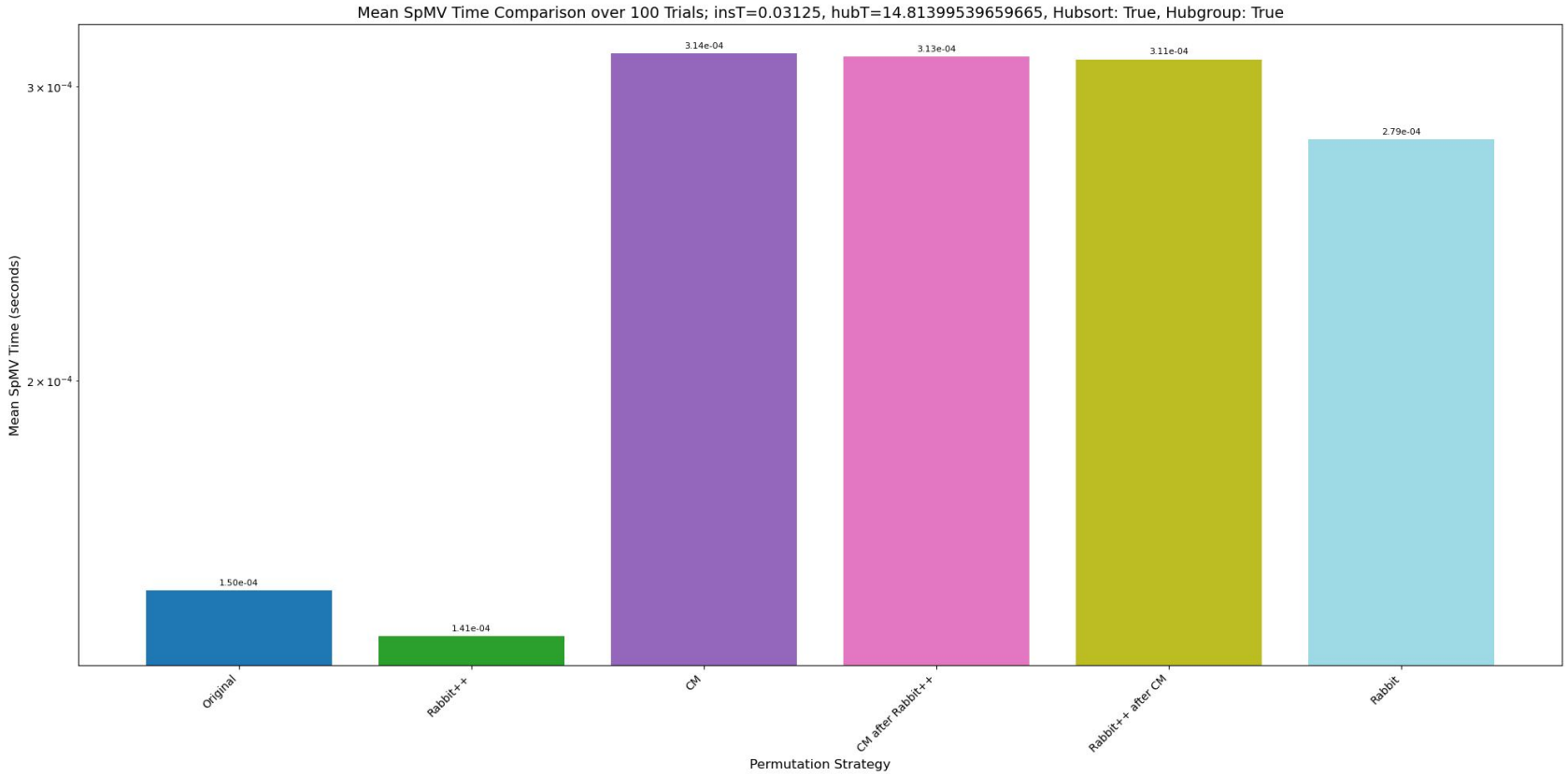
Associated Spy Plots



Can we optimize for Rabbit++ here?



After optimizing for Rabbit++



Blocking

Why is Rabbit Bad with Structure?

With more vertices, it gets harder and harder to classify distant vertices without disturbing already established community structure. If we localize our algorithm's scope to smaller, more cohesive block of the matrix, these distant vertices cannot affect the optimality of our ordering.

Given Time Complexity of Rabbit, can we Grid on dimension with Blocking?

Let’s try to divide the target matrix A into blocks of size block_size.

If this works, how does this make GPU implementation easier? Especially on just-in-time update-dependent operations like merge.

Note: with fixed density=0.0001, increasing dimension to 10e5 yields ***abysmal*** reordering times (>1hr as compared to <2 seconds for 10e4 and under). This study >**100x**’s nnz count per exponential step in dimension -> ~**100x** reordering time complexity.

Blocking Scheme

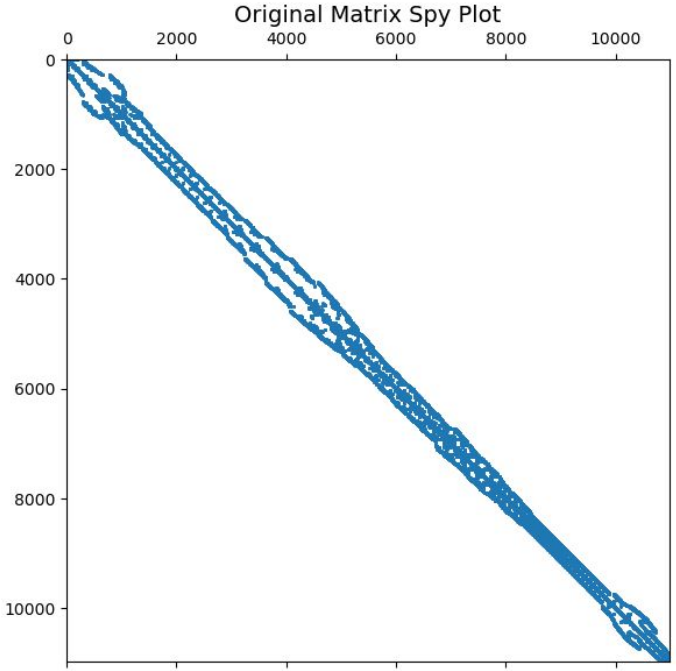
Algorithm 2 Grid-Blocked Rabbit

Require: A, $n \leftarrow \dim(A)$, $d \leftarrow$ desired block size

Ensure: d is a "fittingly-rounded" grid size

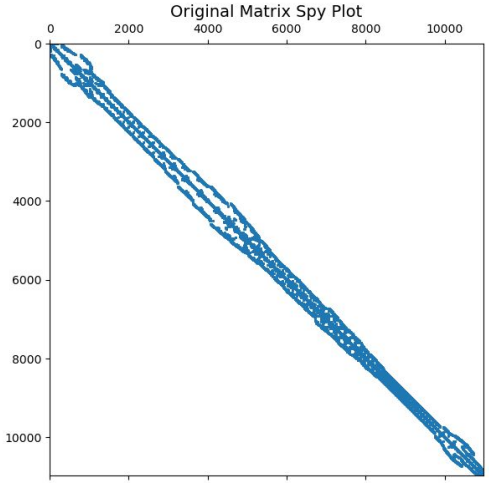
```
n_blocks ← ⌈(n+d−1)/d⌉
P_global ← I                                   ▷ Global permutation starts as identity
for idx ← 1 to n_blocks do
    start ← idx · d
    end ← min(start + d, n)
    block ← A[start : end, start : end]
    P_block ← Rabbit(block)
    P_global[start : end] ← P_global[start : end][P_block]   ▷ Accumulate this
permutation into global
end for
```

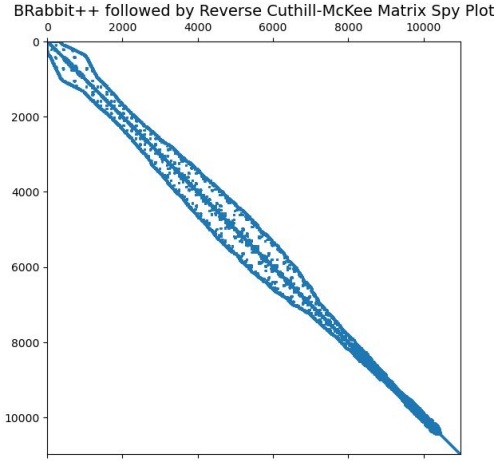
Strongly diagonal bcsstk17: 10974-dimensional Structural Problem

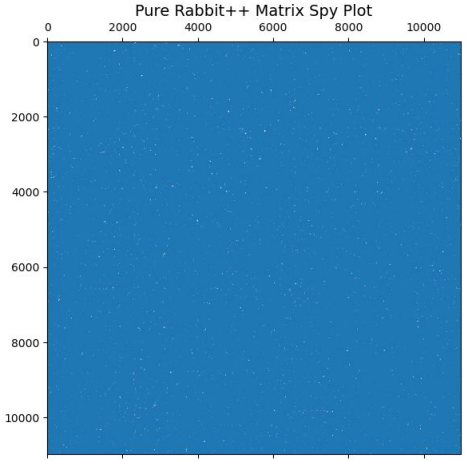


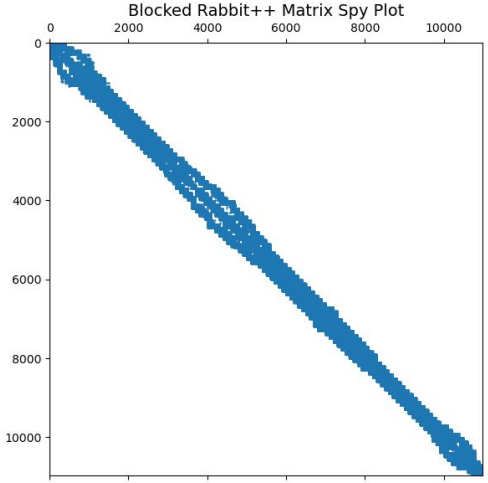
Associated Spy Plots

Plots look largely the same, noise at the ends of the matrix are pushed towards communities

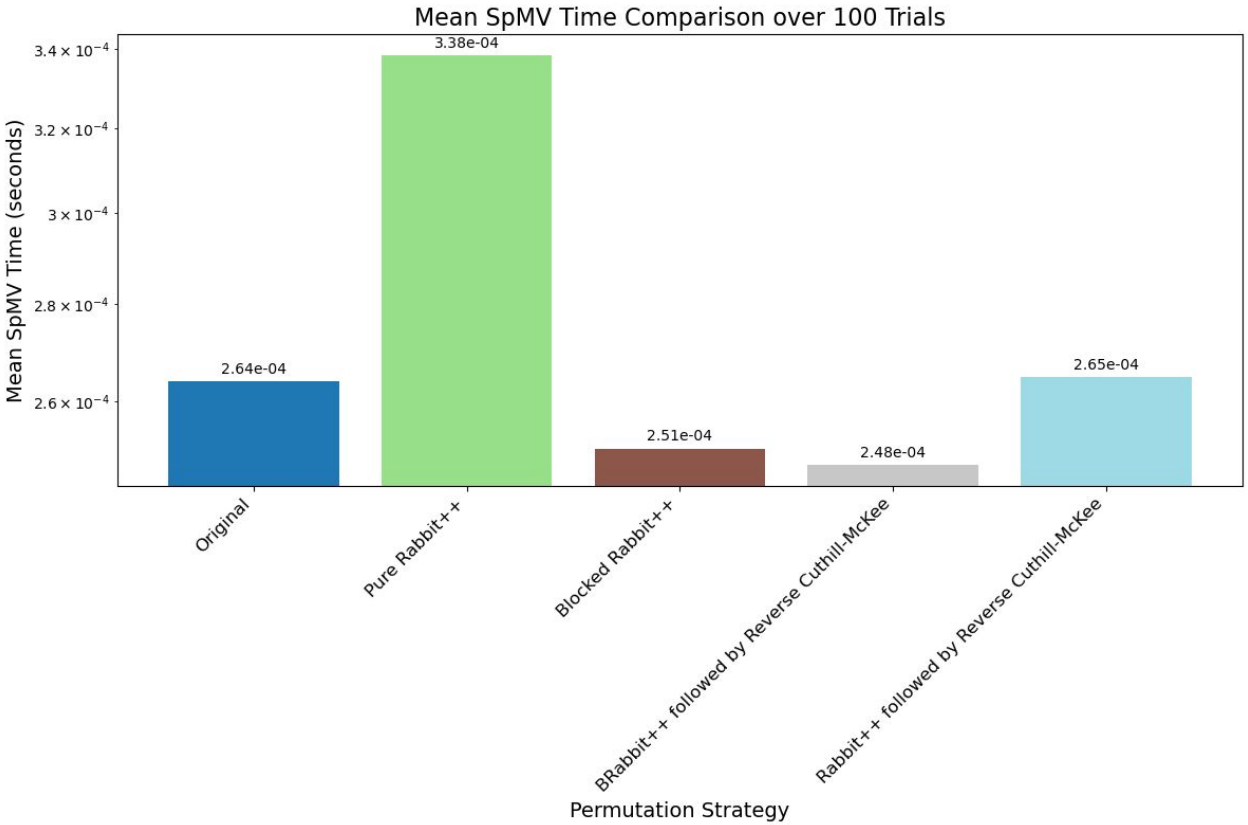




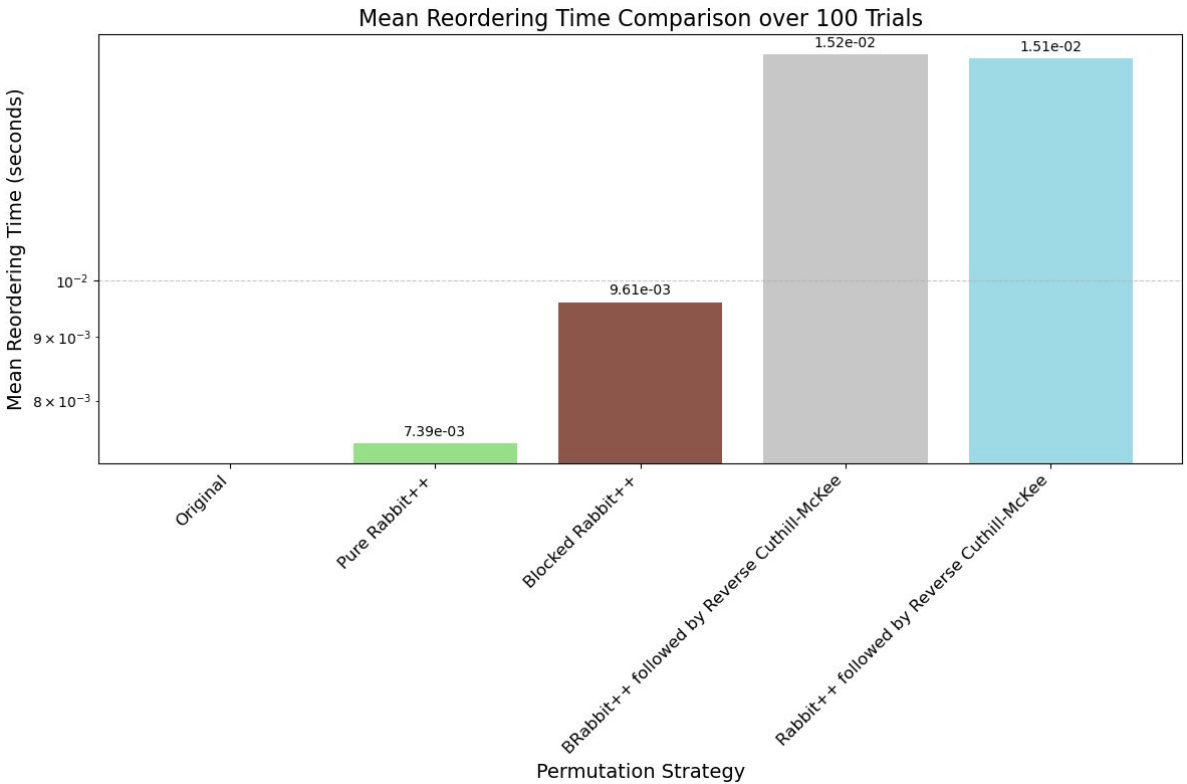




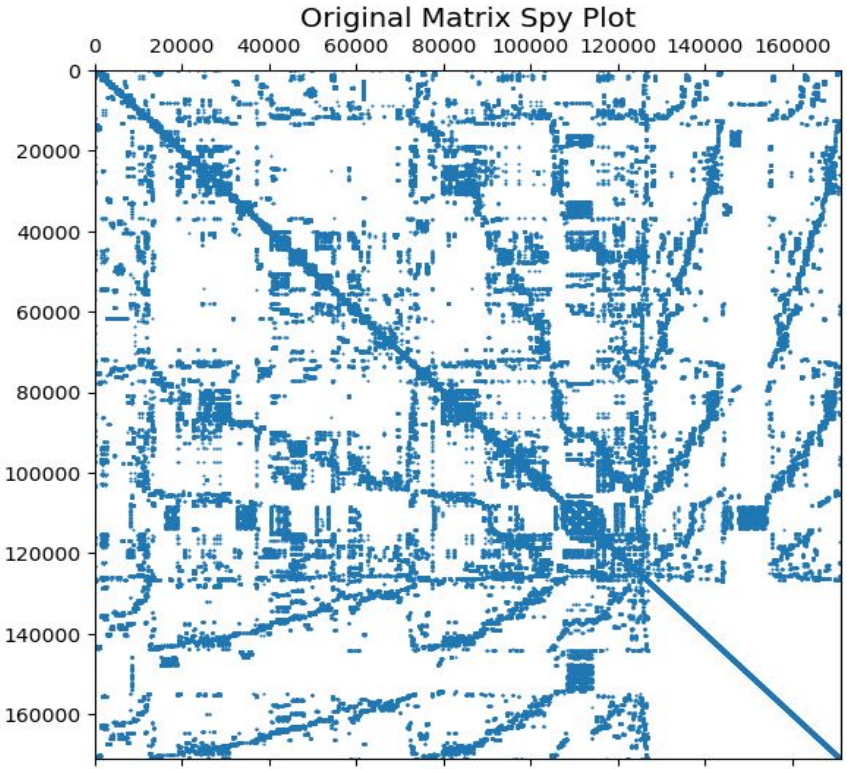
Effects on SpMV Time



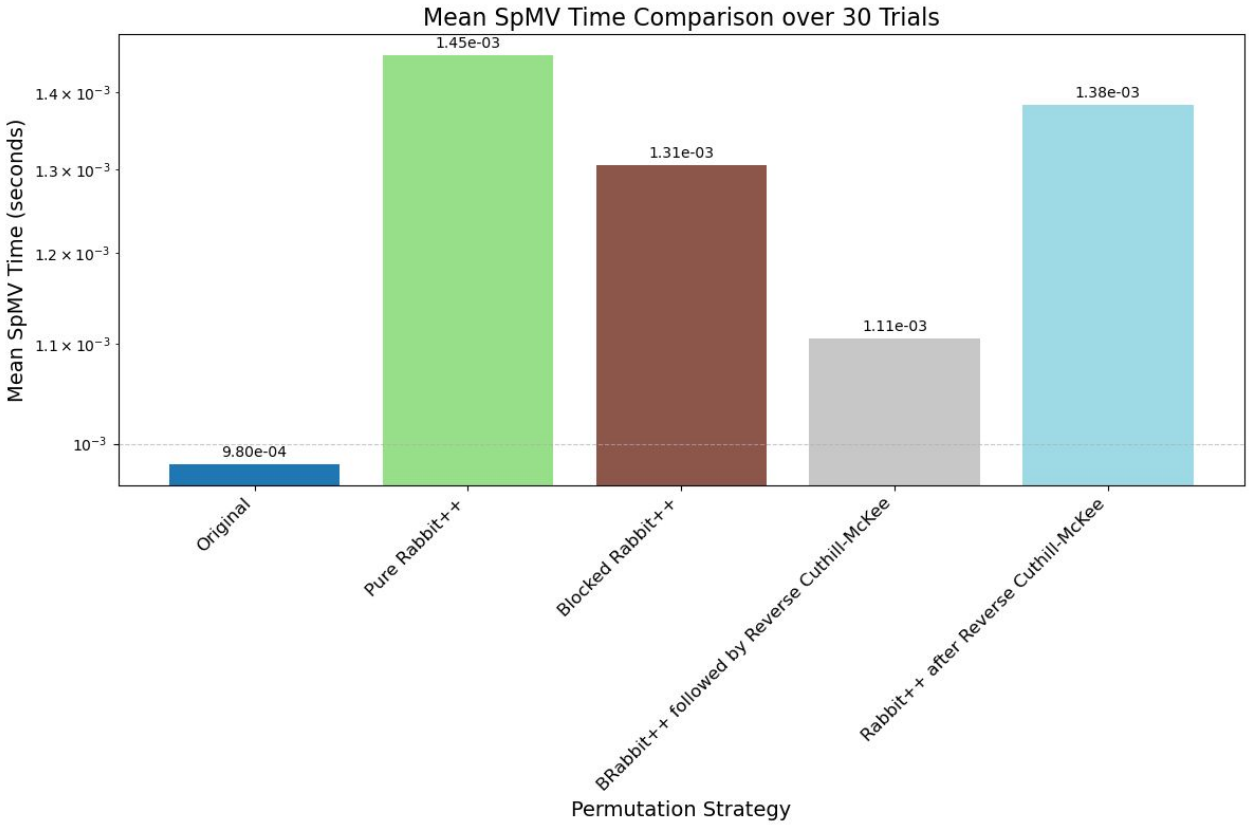
Bcsstk17: Reordering TIme



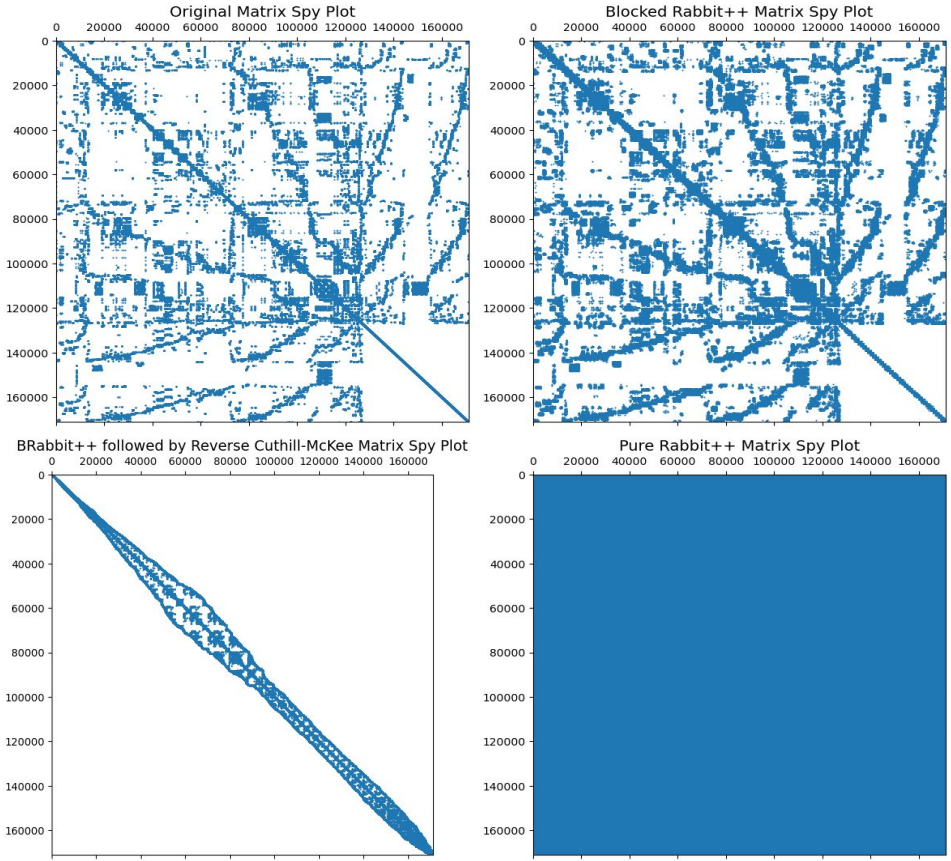
Another Adversarial Example: scircuit



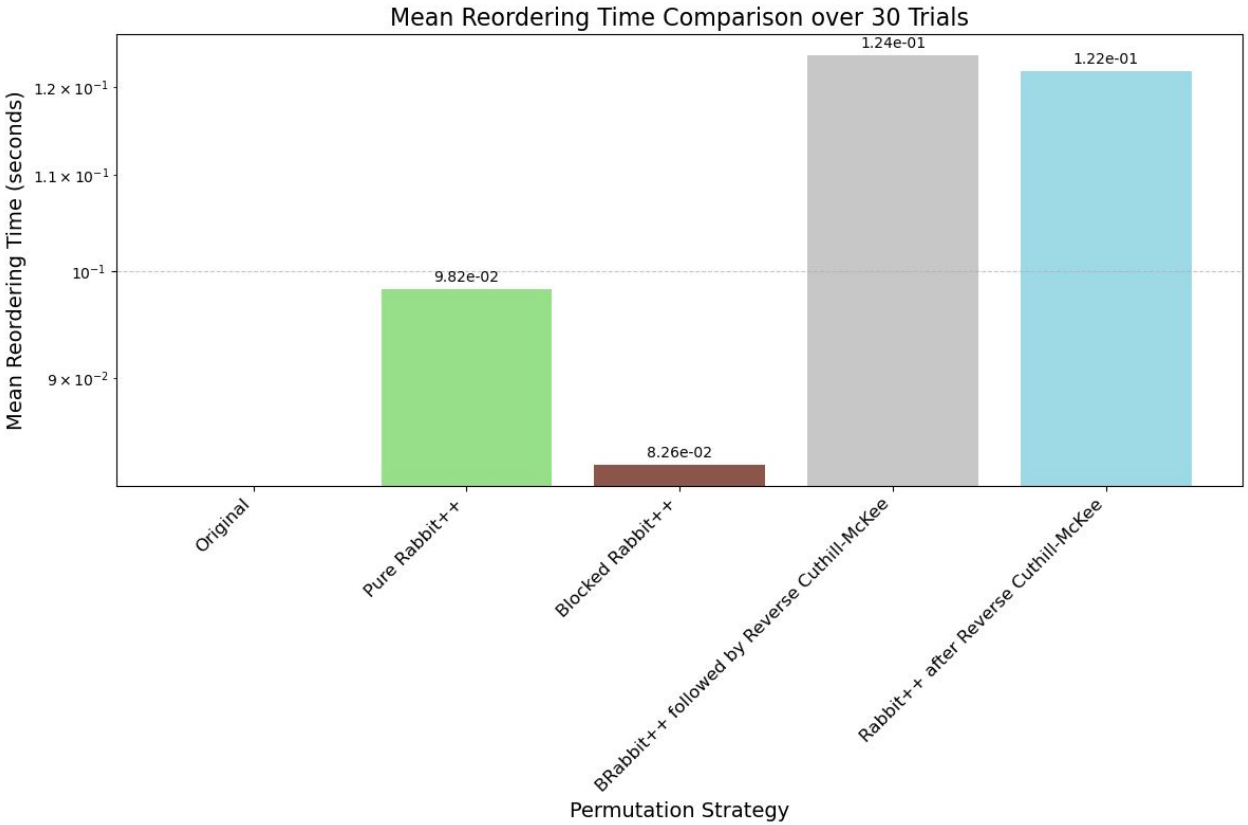
Effects on SpMV Time



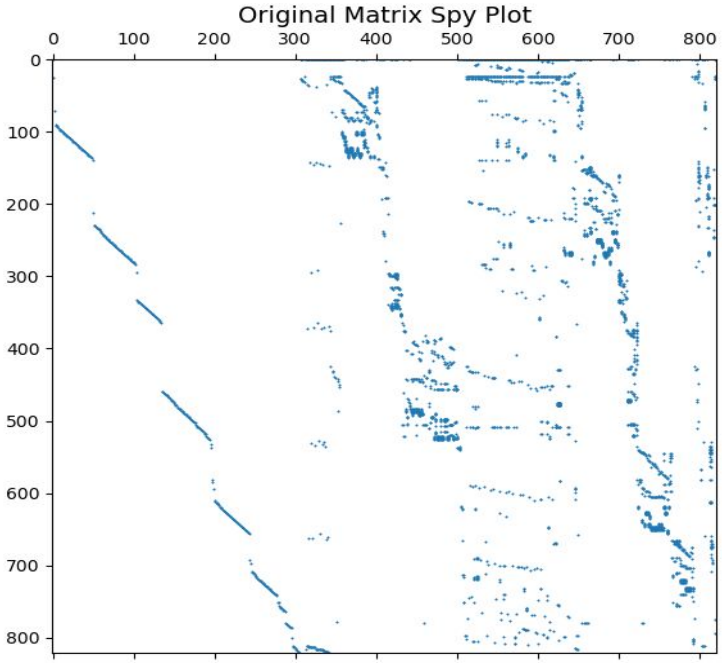
Associated Spy Plots



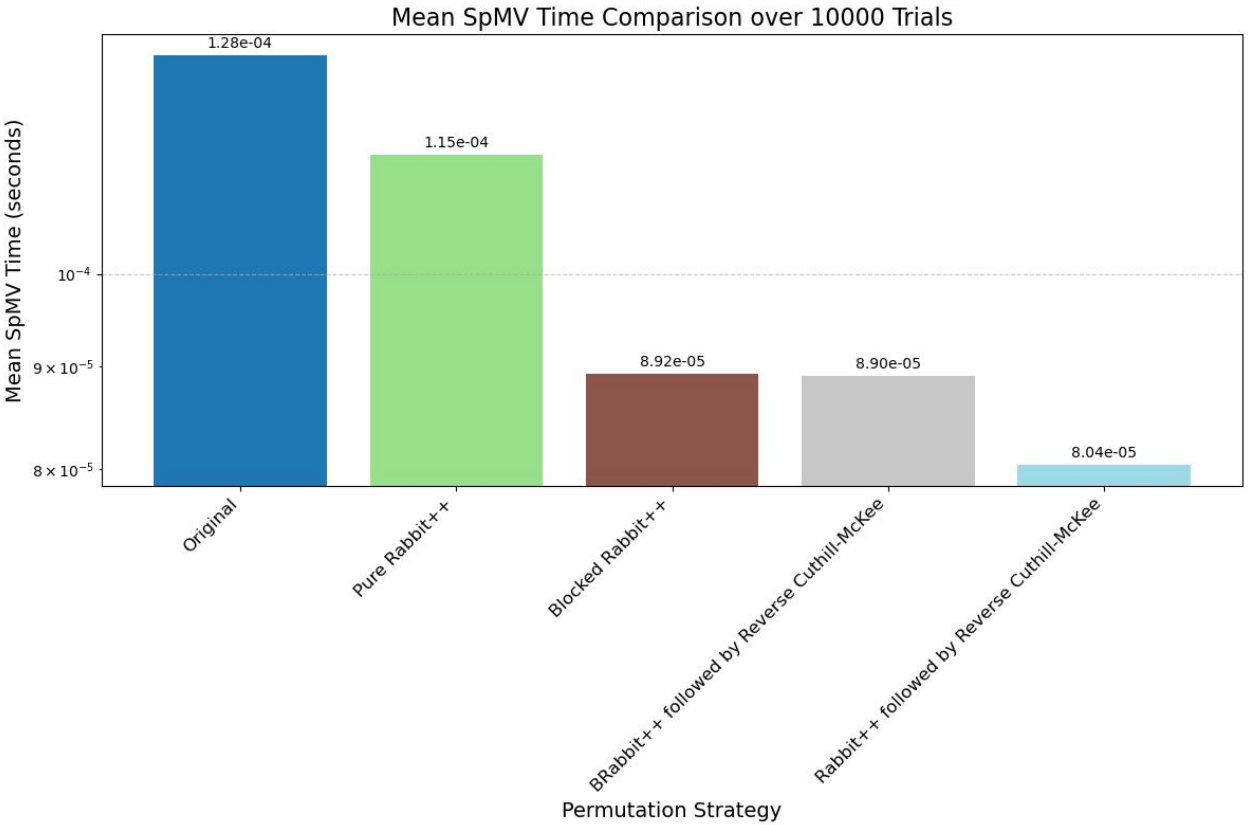
scircuit: Reordering Times



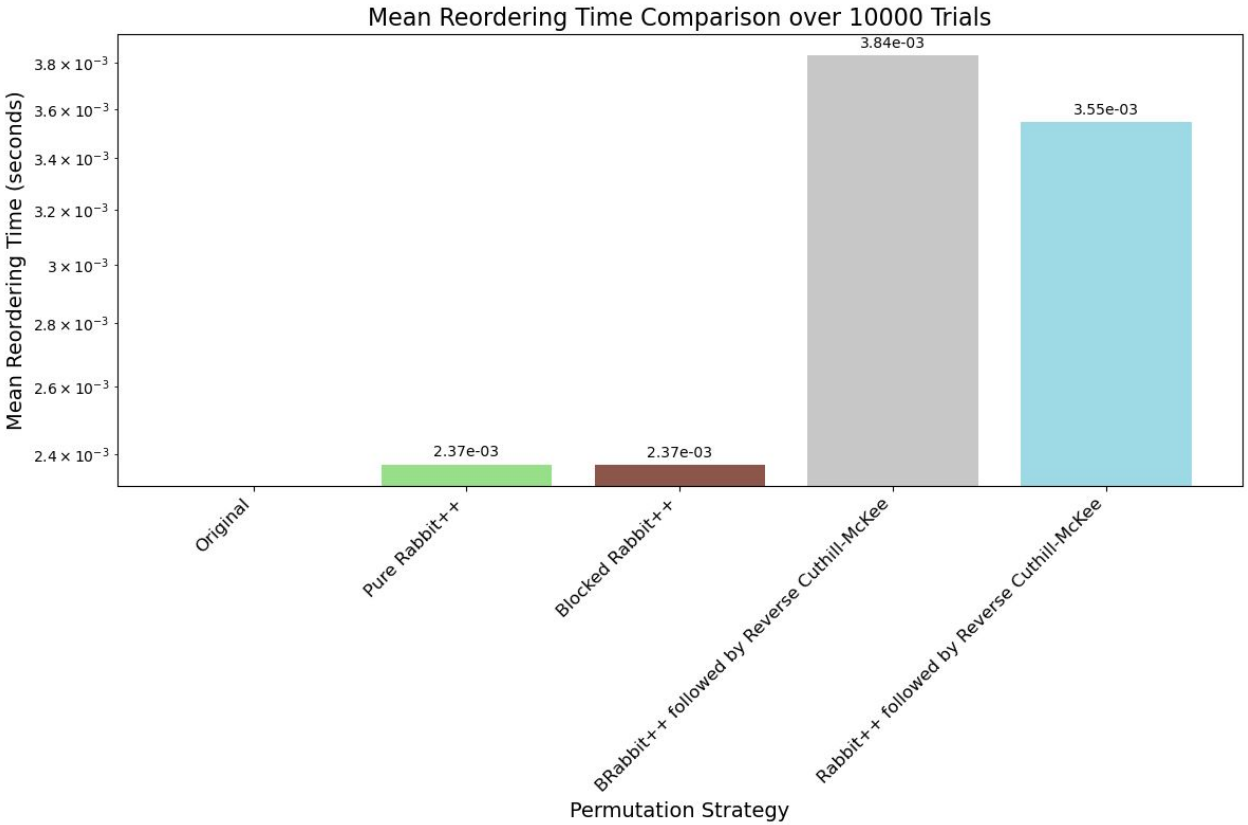
What about on a Rabbit++ Friendly Example: bp0



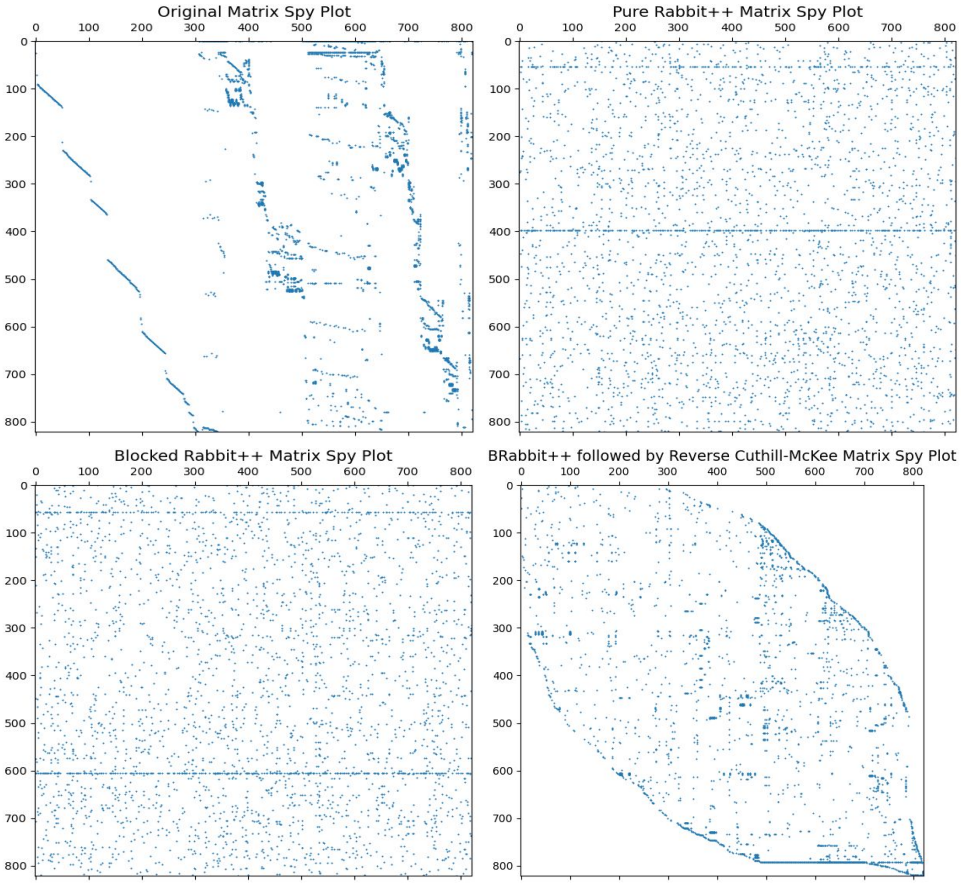
SpMVs are Markedly Faster



bp0: Reordering Times



Associated Spy Plots



Ongoing Work

Problem Space Potential

1. Adaptive Block Size
   a. Heuristic on sparsity, or on intercommunity minimization
2. Adaptive Parameter Search
   a. Heuristic on far entries/intercommunity edges determined by clustering (k-means) to determine max_levels or other Rabbit++ parameters
   b. Can do per-block parameter search
3. Adaptive Stacking
   a. Can artificially tighten subcommunity linking for faster time to optimal Rabbit++
4. Can combine the above

How Can We Quantify when Stacking is Favorable?

Stacking can be very powerful due to quick runtime and modest performance gain given the correct structural conditions.

How do we know when we should apply stacking and at what point?

We know heuristically, stacking will probably help when structure is induced, in terms of density, “tightening” the matrix (either in bandwidth or king-cluster proximity), etc.

We must gain understanding on what explicit conditions on a subject matrix induce a favorable matrix for Rabbit++, in terms of reorder time or SpMV runtime.

J. Arai et al. C++ Paper Parallelization and Implementation

Vertex now contains attributes in atomic sub-struct “atom” to avoid contention in data races:

1. Strength, atomic float
2. child, atomic int
3. Sibling, atomic int

Parallelized:

- FindBestDestination
- Merge

```
Toplevel ← ∅                                          ▷ Set of top-level vertices
▷ Incrementally aggregate vertices
for each u ∈ V in increasing order of degree do in parallel
  degree_u ← UINT64_MAX                               ▷ Set an invalid degree
  AtomicSwap(degree_u, atom[u].degree)                ▷ Invalidate u
  v ← FindBestDestination(u)
  if ΔQ(u, v) ≤ 0 then                                ▷ If the modularity does not improve
    atom[u].degree ← degree_u                         ▷ Restore the value
    Toplevel ← Toplevel ∪ {u}                         ▷ u is a new top-level
    continue                                          ▷ Move to the next vertex
  atom_v ← AtomicLoad(atom[v])
  if atom_v.degree ≠ UINT64_MAX then                  ▷ Check validity
    sibling[u] ← atom_v.child
    atom'_v.degree ← atom_v.degree + degree_u
    atom'_v.child ← u
    if CAS(atom[v], atom_v, atom'_v) then
      dest[u] ← v                                     ▷ u is successfully merged to v
      continue                                        ▷ Move to the next vertex
  sibling[u] ← UINT32_MAX                             ▷ Rollback
  atom[u].degree ← degree_u                           ▷ Rollback
  Save u to retry merge later
```

Thank you!